

PROJECT AND TEAM INFORMATION

PriorityFIX

(Try to choose a catchy title. Max 20 words).

PriorityFIX: A Priority-Based Civic Hazard Tracker

Student / Team Information

<i>Team Name:</i> <i>Team #</i>	<i>Avengers</i>
Team member 1 (Team Lead) (Walia Shubhra, 2510012086, 2028163, waliashubhra14@gmail.com):	<i>Shubhra Walia -</i> <i>7017629692</i> <i>waliashubhra14@gmail.com</i> 
Team member 2 (Khanna Navi, 2510370510, 2028780, navikhanna2007@gmail.com):	<i>Navi Khanna -</i> <i>9888949989</i> <i>Jsnow789@xyz.com</i> 
Team member 3 (Makhetha Tsep0, 252029024, 25022043, tsep0-makhetha1901@gmail.com):	<i>Tsep0 Makhetha -</i> <i>7895850475</i> 

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

(Describe the problem you want to solve and why it is important. Max 300 words).

Municipal corporations receive an overwhelming volume of civic complaints from residents, spanning everything from minor potholes to critical hazards such as major water pipeline bursts and exposed live wires. Because these complaints are typically logged and handled in the order they arrive, urgent, safety critical issues often wait behind minor ones, leading to delayed fixes for hazards that put public safety at risk. This matters because the cost of delay is not uniform: a small pothole and a burst pipeline are not equally urgent, yet a first-come-first-served system treats them as if they were. A tool that automatically ranks incoming complaints by severity would let authorities focus limited repair crews on what genuinely needs attention first, reducing the risk of accidents and prolonged public inconvenience. Officials also currently lack a consolidated, at-a-glance view of how many complaints are open, in progress, or resolved, and duplicate reports of the same underlying issue are often logged separately, further cluttering an already overwhelmed queue.

State of the Art / Current solution (1 pt)

(Describe how the problem is solved today (if it is). Max 200 words).

Today, most municipal complaint systems record grievances in the order they come in. They handle them mostly on a first-come-first-served basis. Staff members who are already overwhelmed do the triage. When digital portals exist they usually only act as a way to submit complaints and track their status. These portals don't function as systems that prioritize issues by importance. Complaints are rarely re-ranked based on how serious they're. There is no automated process to make sure urgent cases like flooding or broken streetlights get handled before ones, like a small pothole or a chipped sign. This causes response times. In the situations dangerous problems stay unaddressed while less important tickets are closed first. These portals also rarely check whether an incoming complaint duplicates one already on file, so the same pothole or burst pipe can generate several redundant tickets that further dilute staff attention, and they rarely offer a dashboard-style summary of overall complaint masses.

Project Goals and Milestones (2 pts)

(Describe the project general goals. Include initial milestones as well any other milestones. Max 300 words).

Design a Ticket capturing a unique ID, category (pothole, pipeline burst, exposed wiring), location, description, timestamp, and a computed severity score, fully viewable on request. Store complaints in a manually implemented linked list (not STL) supporting traversal/display, search by ID, filtering by category, status or location, and update/delete. Rank and serve tickets via a manually implemented binary min-heap (not STL) with peek and extract-highest-priority operations, heapify-up on insertion, heapify-down on extraction, and the rule that a lower score means higher urgency. Define clear severity levels per hazard type, use the score as the heap key, and support priority recalculation with re-heapifying when conditions change. Track status so tickets start OPEN and move to InPROGRESS then RESOLVED, viewable and filterable by status. Detect likely duplicates by comparing category and location, warn the user, and let them review the existing ticket before creating a new one. Provide a GUI dashboard showing total complaints, status counts, the top-priority unresolved complaint, and the full ranked list, with access to registration, search, status updates and ticket viewing. Generate reports by category, status, severity distribution, and totals.

Project Approach (3 pts)

(Describe how you plan to articulate and design a solution. Including platforms and technologies that you will use. Max 300 words).

The Complaint (Ticket) system records each grievance with a unique auto-generated ID, category (e.g., pothole, pipeline burst, exposed wiring), location, description, timestamp, and a numeric severity score indicating urgency. It utilizes a manually implemented `LinkedList` of Complaint nodes for ticket management, allowing insertion, traversal, lookup by Ticket ID, and filtering by various criteria. A `PriorityQueue`, crafted as a binary min-heap, prioritizes tickets based on severity, implementing heapify-up and heapify-down operations to maintain heap validity and urgency rules. The `TicketingSystem` class coordinates these structures, managing ticket states (`OPEN`, `InPROGRESS`, `RESOLVED`), preventing duplicates, and synchronizing linked list and heap updates. A Dashboard/GUI provides insights into complaint metrics and access to functionalities like registration and ticket viewing. Developed in C++ with a graphical frontend using VS Code, it emphasizes custom data structure implementation and Git for version control, thereby ensuring collaborative efficiency and error management.

System Architecture (High Level Diagram)(2 pts)

(Provide an overview of the system, identifying its main components and interfaces in the form of a diagram using a tool of your choice).

[Complaint/Ticket] stored in- > [LinkedList (ticket store)] [LinkedList] feeds > [PriorityQueue / Min-Heap (severity ranking)] [PriorityQueue] serves next ticket to > [TicketingSystem] [TicketingSystem] updates status in > [LinkedList] [TicketingSystem] outputs > [Authority Dashboard]

Project Outcome / Deliverables (1 pts)

(Describe what are the outcomes / deliverables of the project. Max 200 words).

The primary deliverable is a working C++ application with a graphical dashboard implementing the priority-based complaint tracker: a manually implemented linked-list ticket store, a manually implemented heap-based priority queue for severity-based dispatch, status tracking for each complaint, duplicate-complaint detection, and a reports/statistics view. Secondary deliverables: source code repository, this proposal, a system architecture diagram, and a live demo showing complaint submission with duplicate checking, priority-based retrieval, status updates, and the dashboard and reports views.

Assumptions

(Describe the assumptions (if any) you are making to solve the problem. Max 100 words)

The project scope only uses Data Structures and OOP ideas such as linked lists, priority queues (min-heaps), and the classes needed to support them; the core Linked List and Priority Queue must be implemented manually rather than using STL equivalents. Advanced features such as maps, mobile apps and automated dispatch are put off for later parts of the project. The application includes a graphical dashboard, but is assumed to run as a single-user desktop program without network or multi-user connections.

References

(Provide a list of resources or references you utilised for the completion of this deliverable. You may provide links).

TCS302 DataStructureswithC, course syllabus,GraphicEra (Deemed tobeUniversity), Dehradun .TCS307 ObjectOrientedProgrammingwithC++,coursesyllabus,GraphicEra(Deemedtobe Univer-sity),Dehradun. Cormen,T.H., etal., "IntroductiontoAlgorithms" reference forcircular linked listsandthe Josephusproblem. cppreference.com C++StandardLibrarydocumentationforqueue, stack, andpriorityqueue